

엔터프라이즈 클라우드 플랫폼 엔지니어링 포트폴리오

AWS Landing Zone · Terraform · Private EKS

아키텍처 설계 및 구현: 작성자 · 문서화 보조: OpenAI Codex

2026-08-12

상세 목차

1	프로젝트 개요	3
1.1	포트폴리오가 답하는 질문	3
1.2	숫자의 의미	3
1.3	중심 범위와 단순화	3
2	증거와 완료 상태를 읽는 방법	4
2.1	네 단계의 완료 상태	4
2.2	현재 주장하는 것	4
2.3	현재 주장하지 않는 것	4
3	Target Cloud Architecture	5
3.1	Organization과 account 역할	5
3.2	Private spoke 원칙	5
3.3	Workforce access	5
4	Landing Zone	6
4.1	IPAM과 Transit Gateway	6
4.2	Routing domain	6
4.3	다른 account와의 통신	6
5	서비스 네트워크와 IP 설계	7
5.1	서비스별 CIDR	7
5.2	7개 private subnet tier	7
5.3	Commerce prod 예시	7
6	Terraform 구조와 변경 관리	8
6.1	Root와 state ownership	8
6.2	Reusable module 17개	8
6.3	적용 순서	8
6.4	변경 gate	8
7	Private EKS와 Kubernetes Platform	9
7.1	Foundation state	9
7.2	환경별 node와 보존	9
7.3	Kubernetes platform state	9
7.4	남은 production gap	9
8	Monitoring, Security and Governance	10
8.1	Monitoring flow	10
8.2	Security controls	10
8.3	Governance	10
9	Operations and FinOps	11
9.1	Backup, scheduler와 patch	11
9.2	FinOps	11
9.3	운영 보고서	11
10	협업과 승인 경계	12
10.1	전문 역할	12
10.2	승인 흐름	12

11	저장소 탐색	13
11.1	최소화된 구조	13
11.2	질문별 시작 위치	13
11.3	PDF 재생성	13
12	다음 단계	14
12.1	정리 결과 (2026-08-12)	14
12.2	현재 가장 큰 위험	14
12.3	실행 순서	14
12.4	Production promotion gate	14

1 프로젝트 개요

이 포트폴리오는 AWS 리소스를 단순 나열하지 않고, 기업 환경에서 account, network, Terraform state와 EKS 운영 경계를 어떻게 나눌지 설명합니다. 중심 주제는 AWS Organizations와 Landing Zone, 서비스별 IP 계획, reusable Terraform, Private EKS와 production 준비도입니다.

1.1 포트폴리오가 답하는 질문

- AWS account와 OU는 어떤 책임으로 나누는가?
- 다른 account와 workload VPC는 Landing Zone TGW를 통해 어떻게 통신하는가?
- 서비스마다 다른 IP 수요와 성장 여유를 어떻게 반영하는가?
- Terraform root, module과 state는 어떤 기준으로 분리하는가?
- EKS control plane, Node, Pod와 application network는 어떻게 분리하는가?
- 코드 구현과 실제 production 완료를 어떤 증거로 구분하는가?

1.2 숫자의 의미

표시	실제 의미	완료로 오해하면 안 되는 내용
환경 3	dev, stg, prod 공통 platform 환경	AWS account 3개가 실제 생성됐다는 뜻이 아님
서비스 5	commerce, payments, analytics, customer-profile, internal-admin	실제 업무 서비스가 배포됐다는 뜻이 아님
VPC 15	서비스 5 × 환경 3의 독립 Terraform root	AWS에서 생성 완료된 VPC 수가 아님
subnet tier 7	LB, AP, DB, Node, Pod, TGW, EKS Cluster	public subnet을 포함하지 않음
module 17	.tf가 있는 reusable module	module 수 자체가 품질 지표는 아님

1.3 중심 범위와 단순화

본문 중심

AWS Organizations / Landing Zone / Service Network
Terraform root / module / state
Private EKS / Security / Monitoring / Operations / FinOps

이번 정리에서 제외

Agent Runtime / config / JSON schema / simulation fixture
프로젝트 전용 validation·운영 script와 CI
비교용 PDF builder와 확장 AI Platform 문서

Agent는 실행 시스템이 아니라 전문 역할을 나누어 검토하는 문서형 협업 모델로만 남겼습니다.

2 증거와 완료 상태를 읽는 방법

2.1 네 단계의 완료 상태

단계	판단 기준	이 저장소의 예
Code	Terraform이나 문서가 존재하고 정적 검토 가능	VPC, TGW, EKS와 Backup module
Reviewed	format, validate, policy·보안 검토 결과 있음	다음 sandbox 절차에서 새로 기록할 대상
Deployed	실제 account에서 plan/apply와 post-check 완료	현재 증거 없음
Operational Evidence	장애, 복구, 비용과 SLO가 기간·원본과 연결됨	현재 증거 없음

코드가 있다는 이유로 AWS에 배포됐다고 표현하지 않습니다. Target 설계와 placeholder도 실제 성과로 합산하지 않습니다.

2.2 현재 주장하는 것

- Organizations, OU, SCP와 Tag Policy 코드가 있습니다.
- IPAM, TGW hub, service VPC와 route ownership이 코드로 분리되어 있습니다.
- 5개 서비스의 dev/stg/prod CIDR와 7개 subnet tier가 설계되어 있습니다.
- 공통 environment foundation과 Kubernetes platform state가 분리되어 있습니다.
- Private EKS, managed add-on, logs와 Kubernetes platform 코드가 있습니다.
- 보안, 백업, patch, budget과 비용 이상 탐지 코드가 있습니다.

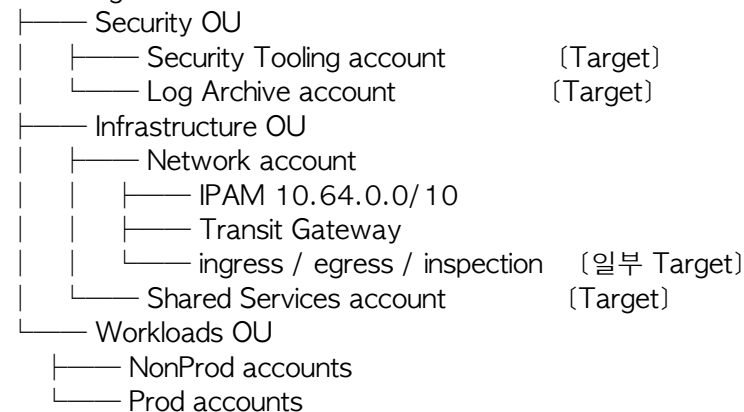
2.3 현재 주장하지 않는 것

- 모든 Terraform root의 실제 account plan/validate 성공
- production apply와 실제 service traffic
- 중앙 inspection, immutable log archive와 WAF association 동작
- EKS autoscaling, upgrade, drain과 restore 성공
- availability, MTTR, 비용 절감과 audit 성과

3 Target Cloud Architecture

3.1 Organization과 account 역할

AWS Organizations

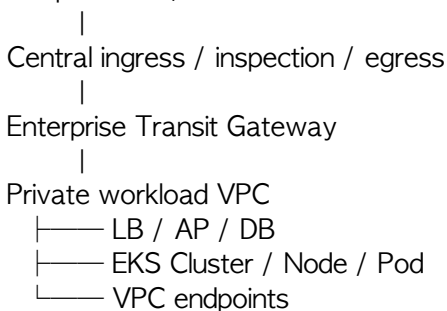


Terraform은 OU와 정책을 구성하지만 account vending은 포함하지 않습니다. 실제 account ID, delegated administrator, RAM principal과 중앙 log destination은 배포 입력이 필요합니다.

3.2 Private spoke 원칙

Workload VPC에는 public subnet, Internet Gateway와 NAT Gateway를 직접 만들지 않습니다. north-south traffic, 중앙 inspection, 다른 account와의 east-west traffic은 Landing Zone TGW를 통과합니다.

On-premises / External



중앙 ingress/egress attachment와 실제 return route는 아직 Target입니다. Workload route가 TGW를 향한다는 것만으로 외부 통신이 완성되지는 않습니다.

3.3 Workforce access

사람의 접근은 Corporate IdP → IAM Identity Center → Permission Set → AWS Account 흐름을 목표로 합니다. 현재는 설계 문서이며 permission set과 account assignment Terraform은 아직 없습니다.

4 Landing Zone

4.1 IPAM과 Transit Gateway

Component	State owner	현재 코드 범위
IPAM	landing-zone/ipam	기업 CIDR와 서비스별 pool
TGW hub	landing-zone/network-hub	ASN 64520, route-table domain과 RAM share
Service attachment	services/<service>/<env>	private VPC와 TGW attachment
Connectivity	landing-zone/connectivity	association과 명시적 허용 route

4.2 Routing domain

Enterprise TGW

—— nonprod route table	dev / stg attachment
—— prod route table	prod attachment
—— shared route table	DNS / shared service
—— inspection route table	firewall path (Target)

TGW default association과 propagation은 비활성입니다. attachment가 생성되었다고 다른 VPC와 자동으로 통신하지 않으며, connectivity state가 허용된 association과 route를 소유합니다.

4.3 다른 account와의 통신

1. Network account가 TGW와 RAM share를 소유합니다.
2. Workload account의 service root가 share된 TGW에 VPC를 attachment합니다.
3. Connectivity root가 environment domain에 attachment를 association합니다.
4. 허용된 destination만 TGW route로 등록합니다.
5. VPC route table은 subnet 용도에 맞는 destination을 TGW로 전달합니다.
6. 중앙 inspection과 return route가 실제 account에서 검증되어야 통신 완료로 인정합니다.

이 분리는 service team이 중앙 route-table policy를 임의 변경하지 못하게 하고 Network team이 모든 service VPC state를 소유하는 문제도 피합니다.

5 서비스 네트워크와 IP 설계

5.1 서비스별 CIDR

Service	Supernet	Dev	Stg	Prod
commerce	10.64.0.0/13	10.64.0.0/20	10.64.32.0/19	10.65.0.0/16
payments	10.72.0.0/14	10.72.0.0/22	10.72.8.0/21	10.73.0.0/18
analytics	10.76.0.0/14	10.76.0.0/20	10.76.64.0/18	10.77.0.0/16
customer-profile	10.80.0.0/14	10.80.0.0/22	10.80.8.0/21	10.81.0.0/19
internal-admin	10.84.0.0/16	10.84.0.0/22	10.84.4.0/22	10.84.16.0/20

commerce와 analytics는 Pod와 batch 확장 여유를 크게 잡았습니다. payments와 customer-profile은 중요도와 보안 경계는 높지만 주소 소비량은 더 작게 가정했습니다. internal-admin은 내부 사용자 중심으로 가장 작은 범위를 배정했습니다.

5.2 7개 private subnet tier

Tier	Prefix 원칙	역할
Pod	VPC prefix +3	VPC CNI secondary ENI, 가장 큰 pool
Node	+4	managed node primary ENI
AP	+5	application ENI
DB	+6	database, cache와 stateful data
LB	+6	내부 ALB/NLB ENI
TGW	/28	TGW attachment 전용
EKS Cluster	/28	control-plane x-ENI 전용

DB subnet에는 internet default route를 두지 않습니다. TGW와 EKS Cluster subnet도 전용 목적만 갖도록 작게 분리합니다.

5.3 Commerce prod 예시

Tier	AZ-1	AZ-2	AZ-3
Pod	10.65.0.0/19	10.65.32.0/19	10.65.64.0/19
Node	10.65.128.0/20	10.65.144.0/20	10.65.160.0/20
AP	10.65.176.0/21	10.65.184.0/21	10.65.192.0/21
DB	10.65.200.0/22	10.65.204.0/22	10.65.208.0/22
LB	10.65.212.0/22	10.65.216.0/22	10.65.220.0/22
TGW	10.65.255.160/28	10.65.255.176/28	10.65.255.192/28
EKS	10.65.255.208/28	10.65.255.224/28	10.65.255.240/28

Multi-account에서 AZ 이름은 같은 물리 zone을 보장하지 않으므로 apne2-az1, apne2-az2, apne2-az3 AZ ID를 사용합니다.

6 Terraform 구조와 변경 관리

6.1 Root와 state ownership

```

terraform/
├── organization/           organization policy state
├── landing-zone/
│   ├── ipam/              address state
│   ├── network-hub/       TGW hub state
│   └── connectivity/       central route policy state
├── services/<service>/<env>/ service VPC state
├── environments/<env>/      AWS foundation state
└── environments/<env>/platform/ Kubernetes / Helm state
  
```

Organization은 blast radius, Landing Zone은 Network owner, service root는 VPC lifecycle, platform root는 Kubernetes API와 rollback 경계 때문에 분리합니다.

6.2 Reusable module 17개

Group	Modules
Organization	organization, scp-policy
Network	network, service-vpc, transit-gateway-hub, transit-gateway-routing, security-group, route-policy
Composition/Security	workload-environment, security, iam, waf
Platform	eks, kubernetes-platform, observability
Operations/Cost	operations, cost

Root는 환경 값과 조립을, module은 입력·출력과 resource lifecycle을 소유합니다. 빈 예약 module과 실행형 Agent 전용 module은 단순화 과정에서 제거했습니다.

6.3 적용 순서

```

organization
├── ipam
├── network-hub
├── service VPC
└── connectivity
  
```

```

environment foundation
└── Kubernetes platform
  
```

6.4 변경 gate

1. ticket, owner, root/state와 성공·원복 조건을 확인합니다.
2. terraform fmt, init -backend=false, validate를 실행합니다.
3. 실제 account와 backend가 고정된 plan을 생성합니다.
4. replace/destroy, IAM, public exposure, CIDR와 비용을 검토합니다.
5. plan hash와 designated approver를 연결합니다.
6. protected CI/CD가 apply합니다.
7. 동일한 관측 범위로 health, logs, metrics, backup과 비용을 확인합니다.

현재 저장소에는 project-specific validation wrapper와 CI가 없습니다. 깨끗한 sandbox에서 위 절차를 재현한 다음 최소 workflow를 다시 추가합니다.

7 Private EKS와 Kubernetes Platform

7.1 Foundation state

Private EKS 1.35

- └── EKS Cluster subnet control-plane x-ENI
- └── Node subnet managed node primary ENI
- └── Pod subnet VPC CNI secondary ENI
- └── AL2023 managed node group
- └── VPC CNI custom networking + prefix delegation
- └── VPC CNI / EBS CSI Pod Identity
- └── managed add-on
- └── KMS encrypted control / container logs

Cluster creator bootstrap admin은 비활성화하고 명시적인 Access Entry를 사용합니다. API endpoint는 private이며 실제 적용 runner는 VPN, Direct Connect 또는 SSM-connected network path가 필요합니다.

7.2 환경별 node와 보존

환경	AZ	Node	Control log	Container log
dev	2	Spot general 1/1/3	90일	30일
stg	2	On-Demand general 2/2/5	90일	90일
prod	3	system 3/3/6, application 3/3/12	365일	365일

숫자는 min/desired/max입니다. Terraform의 desired-size drift ignore는 Cluster Autoscaler나 Karpenter가 설치되었다는 뜻이 아닙니다.

7.3 Kubernetes platform state

- AZ별 ENIConfig: Pod subnet과 EKS cluster security group 연결
- Istio revision과 strict mTLS
- Prometheus, Alertmanager와 Grafana
- application namespace의 ResourceQuota와 LimitRange
- platform-critical, application-high, batch-low PriorityClass
- typed PDB interface

7.4 남은 production gap

Gap	필요한 증거
HPA/KEDA와 node autoscaler 없음	load 기반 Pod/node scale-out-in
실제 PDB instance 없음	drain과 topology rehearsal
default-deny NetworkPolicy 없음	허용 통신표와 deny test
중앙 장기 log archive 없음	source-to-archive delivery와 query
restore 결과 없음	EKS composite recovery와 RPO/RTO
upgrade 실행 기록 없음	dev→stg→prod와 rollback evidence

8 Monitoring, Security and Governance

8.1 Monitoring flow

EKS control logs / Container Insights / VPC Flow Logs
→ CloudWatch Logs, metrics, alarm, dashboard

Kubernetes metrics → Prometheus → Alertmanager / Grafana

Signal	Current	Gap
EKS control log	5종, KMS, 환경별 retention	중앙 archive와 delivery test
Container log	application/dataplane/host/performance	node coverage와 drop alarm
VPC Flow Logs	CloudWatch와 reject-flow alarm	실제 traffic과 tuning
Prometheus	dev 7일, stg 15일, prod 30일	HA, external receiver와 장기 저장

Alert threshold는 초기 기준이며 실제 service SLO와 2~4주 baseline으로 조정해야 합니다. Critical 통지는 업무 서비스와 다른 provider/network path를 가져야 합니다.

8.2 Security controls

Layer	Current code
Organization	nested OU, SCP와 Tag Policy
IAM	explicit trust, GitHub OIDC subject, audit와 break-glass option
Encryption	KMS rotation, EBS default, EKS secret/node/log encryption
Network	private subnet, endpoint, DB internet route isolation
Detection	GuardDuty, Security Hub, Inspector와 VPC Flow Logs
Edge	WAF managed rule, rate limit과 logging module

WAF ACL은 ingress ARN이 없어 아직 association되지 않았습니다. Organization CloudTrail, Config aggregator와 immutable Log Archive도 account 정보가 필요한 Target입니다.

8.3 Governance

SCP는 권한을 부여하지 않고 최대 권한을 제한합니다. 새 SCP는 Policy-Staging OU에서 작은 account 단위로 검증하고 break-glass rehearsal 뒤에 확대합니다. 실제 account와 exception register가 없으므로 현재 policy를 production 적용 완료로 표현하지 않습니다.

9 Operations and FinOps

9.1 Backup, scheduler와 patch

Area	Current code	Production gate
Backup	태그 기반 plan, 환경별 retention	대상 coverage와 restore drill
Vault Lock	prod option 활성화	legal/retention 승인과 복구 검증
Scheduler	dev/stg office-hours, prod 이중 차단	실제 tag 대상과 실행 log
Patch	SSM patch baseline과 patch group	maintenance window와 compliance
Vulnerability	Inspector integration	finding owner와 remediation SLA

Backup job 성공은 application과 EKS 복구 성공을 의미하지 않습니다. recovery point, 데이터 정합성, service health와 실제 RPO/RTO를 함께 검증해야 합니다.

9.2 FinOps

Environment	Monthly budget example	Cost anomaly threshold
dev	USD 300	USD 50
stg	USD 1,000	USD 100
prod	USD 5,000	USD 300

Terraform은 Budget와 Cost Anomaly Detection을 정의합니다. 금액은 포트폴리오 입력값이며 실제 청구액이 아닙니다. 비용 절감은 제안, 승인, 적용과 invoice에서 확인된 실현 절감을 구분합니다.

9.3 운영 보고서

reports/templates/에는 두 개의 사람용 빈 양식만 둡니다.

- 월간 플랫폼 보고서: 신뢰성, 변경, 보안, 비용과 다음 조치
- 장애 보고서: 영향, timeline, 사실·가설·원인, 복구와 재발 방지

실제 보고서는 Git이 아니라 승인된 ticket 또는 문서 저장소에서 관리하고 원본 log나 고객 데이터를 저장소에 복사하지 않습니다.

10 협업과 승인 경계

10.1 전문 역할

Role	검토 책임
Architecture	account, network, module과 state boundary
Terraform	reusable code와 plan impact
Governance/Security	SCP, IAM, encryption과 residual risk
Monitoring/Operations	signal, backup, lifecycle과 recovery
FinOps	tag, budget, anomaly와 실현 절감
CI/CD	plan artifact와 protected deployment gate
Reviewer	누락된 위험과 evidence 독립 검토
Documentation	README, diagram과 PDF 정합성

이 역할은 별도 실행 Runtime이나 AWS administrator가 아닙니다. 코드와 문서의 품질 관점을 나누기 위한 협업 모델입니다.

10.2 승인 흐름

요구사항과 ticket

- Architecture boundary
- Terraform patch
- Security / Operations / FinOps review
- plan artifact
- independent review
- designated human approval
- protected CI/CD apply
- post-check와 보고

어떤 자동화 도구도 production apply, 재시작, drain, failover, restore 또는 alarm suppression을 사람 승인 없이 실행할 수 없습니다.

11 저장소 탐색

11.1 최소화된 구조

```
cloud-portfolio/
├── README.md           전체 범위와 현재 상태
├── AGENTS.md          역할과 변경 안전 경계
├── terraform/         AWS·Kubernetes IaC
│   ├── organization/
│   ├── landing-zone/
│   ├── services/
│   ├── environments/
│   ├── modules/
│   └── diagrams/
├── docs/              domain 기준과 PDF source
├── reports/templates/ 월간·장애 보고서 양식
├── scripts/pdf/       builder와 verifier
└── enterprise-cloud-portfolio.pdf
```

11.2 질문별 시작 위치

질문	시작 위치
OU와 SCP는 어떻게 구성했는가	terraform/organization/main.tf
TGW와 다른 account 연결은 어디에 있는가	landing-zone/network-hub, connectivity
15개 서비스 VPC는 어떻게 나뉘는가	terraform/services, modules/service-vpc
CIDR와 subnet 근거는 무엇인가	docs/service-network-architecture.md
공통 dev/stg/prod 차이는 무엇인가	terraform/environments/*/main.tf
EKS subnet, logs와 add-on은 어디에 있는가	terraform/modules/eks/main.tf
quota, Istio와 Prometheus는 어디에 있는가	modules/kubernetes-platform/main.tf
남은 production risk는 무엇인가	docs/security-review.md, eks-operations.md
보고서는 어떻게 작성하는가	reports/templates/

AWS 구성도는 terraform/diagrams/aws-infrastructure-diagram.md에서 Mermaid로 확인하고 draw.io 원본으로 편집합니다.

11.3 PDF 재생성

```
docs/portfolio-presentation.md
+ docs/portfolio-presentation-header.tex
→ build_portfolio_presentation_pdf.py
→ enterprise-cloud-portfolio.pdf
→ verify_portfolio_pdf.py
```

중간 PNG는 시스템 임시 디렉터리에 만들며 저장소 내부 tmp/를 사용하지 않습니다.

12 다음 단계

12.1 정리 결과 (2026-08-12)

- AWS Landing Zone, Terraform과 EKS 중심으로 README와 PDF를 다시 구성했습니다.
- Agent Runtime, config, schema, fixture와 관련 CI를 제거했습니다.
- Agent 전용 IAM role, EKS Access Entry와 Kubernetes RBAC도 Terraform에서 제거했습니다.
- 빈 예약 module과 구형 PDF builder를 제거했습니다.
- 운영 보고서는 실제 값이 없는 example 없이 빈 template만 유지합니다.
- 최종 PDF source, builder와 verifier를 각각 하나로 고정했습니다.

12.2 현재 가장 큰 위험

Priority	Gap	완료 evidence
P0	깨끗한 환경의 root validate 결과 없음	root별 init/validate 기록
P0	실제 AWS plan 없음	account/backend/provider가 고정된 plan hash
P1	중앙 ingress/egress와 return route 없음	왕복 traffic와 failover test
P1	EKS runtime 검증 없음	API, node, Pod IP, DNS, storage와 log health
P1	restore 증거 없음	격리 restore와 RPO/RTO
P2	autoscaling/PDB/NetworkPolicy 없음	load, drain과 deny test
P2	중앙 audit archive 없음	전달·보존·검색·무결성 증거

12.3 실행 순서

1. 깨끗한 환경에서 standard Terraform validation 절차를 재현합니다.
2. sandbox에서 dev Landing Zone과 한 개 service VPC plan을 생성합니다.
3. TGW association, route, subnet과 VPC endpoint를 확인합니다.
4. dev EKS를 적용해 private access, Pod IP와 log를 검증합니다.
5. destroy 또는 restore rehearsal로 rollback과 복구 가능성을 확인합니다.
6. 반복 가능한 부분만 최소 CI로 다시 추가합니다.
7. 실제 결과와 evidence reference를 README와 PDF에 반영합니다.

12.4 Production promotion gate

- 승인된 plan, plan hash, ticket와 적용 identity
- maintenance window, approver와 rollback owner
- 배포 후 health, log, metric, backup과 비용 검증
- EKS upgrade, drain/PDB, autoscaling과 restore rehearsal
- Security와 service owner의 residual risk 승인
- Target이나 예시값을 실제 성과로 표현하지 않았다는 최종 검토

이 포트폴리오의 완성은 문서나 module 수가 아니라 실제 account에서 반복 가능한 변경과 복구 증거로 판단합니다.